

Università degli Studi di Firenze
Dipartimento di Ingegneria dell'Informazione (DINFO)

Software Dependability

Project Work – A.A. 2025-26 – Tema 1

Modellazione e Verifica Formale di un Sistema di Interlocking Ferroviario

mediante NuXmv (modelli SMV)

Studente: Mattia Manneschi

Docente: A. Fantechi

Anno Acc.: 2025-26

Indice

1	Introduzione e descrizione del sistema	2
1.1	Topologia della stazione	2
1.2	Itinerari e conflitti	2
1.3	Topologia della stazione	2
1.4	Macchine a stati	3
1.5	Connessione con la mutua esclusione	3
2	Modellazione delle macchine a stati generiche	4
2.1	Macchina scambio	4
2.2	Macchina itinerario	4
3	Istanziamento e collegamento I/O	5
3.1	Struttura del modulo principale	5
3.2	Condizione NOCONFLICT	5
3.3	Azione COMMANDS	6
3.4	Condizioni STABLE e FAILED	6
4	Verifica della correttezza statica	6
4.1	Risultato	7
4.2	Motivazione	7
5	Simulazione del modello	7
5.1	Scenario 1 – Sequenza base: AD poi AB (riuso degli scambi)	7
5.2	Scenario 2 – Fallimento di uno scambio e retry (CB)	7
5.3	Scenario 3 – Prenotazione parallela (AB CD)	8
5.4	Scenario 4 – AB bloccata da AD, poi servita	8
6	Verifica formale delle proprietà	9
6.1	Safety-1 (No-collision)	9
6.2	Safety-2 (No-derailment)	9
6.3	Liveness (possibilità di prenotazione)	10
6.4	Fairness (assenza di ritardo indefinito)	10
6.4.1	Analisi del controesempio	11
6.4.2	Causa del fallimento: sistema senza memoria	12
6.4.3	Osservazione	12
7	Riepilogo dell’effort	12
8	Conclusioni	12

1 Introduzione e descrizione del sistema

L'elaborato riguarda la modellazione e la verifica formale di un sistema di **interlocking ferroviario** mediante lo strumento **NuXmv** e modelli SMV (Symbolic Model Verifier).

1.1 Topologia della stazione

La stazione modellata è composta da quattro punti di accesso/uscita (A, B, C, D), quattro scambi (s1, s2, s3, s4) e quattro itinerari possibili. I treni viaggiano da sinistra (A, C) verso destra (B, D).

1.2 Itinerari e conflitti

La tabella seguente riassume gli itinerari, le posizioni degli scambi richieste e i conflitti tra di essi.

Itinerario	Scambi richiesti	In conflitto con
A-B	s1:NORMALE, s2:NORMALE	A-D, C-B
A-D	s1:NORMALE, s2:ROVESCIO, s4:ROVESCIO	A-B, C-B, C-D
C-B	s1:ROVESCIO, s2:NORMALE, s3:ROVESCIO	A-B, A-D, C-D
C-D	s3:NORMALE, s4:NORMALE	A-D, C-B

Tabella 1: Itinerari, posizioni degli scambi e conflitti

Gli itinerari A-B e C-D non sono in conflitto e possono essere prenotati contemporaneamente.

1.3 Topologia della stazione

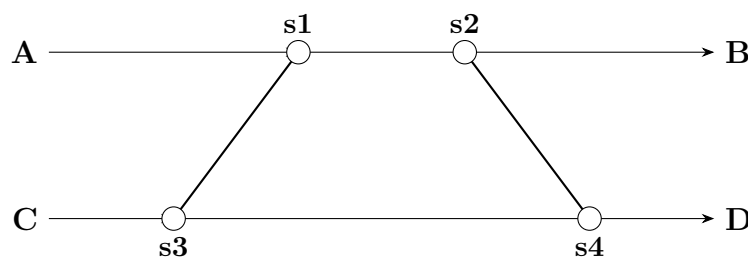


Figura 1: Topologia della stazione ferroviaria

1.4 Macchine a stati

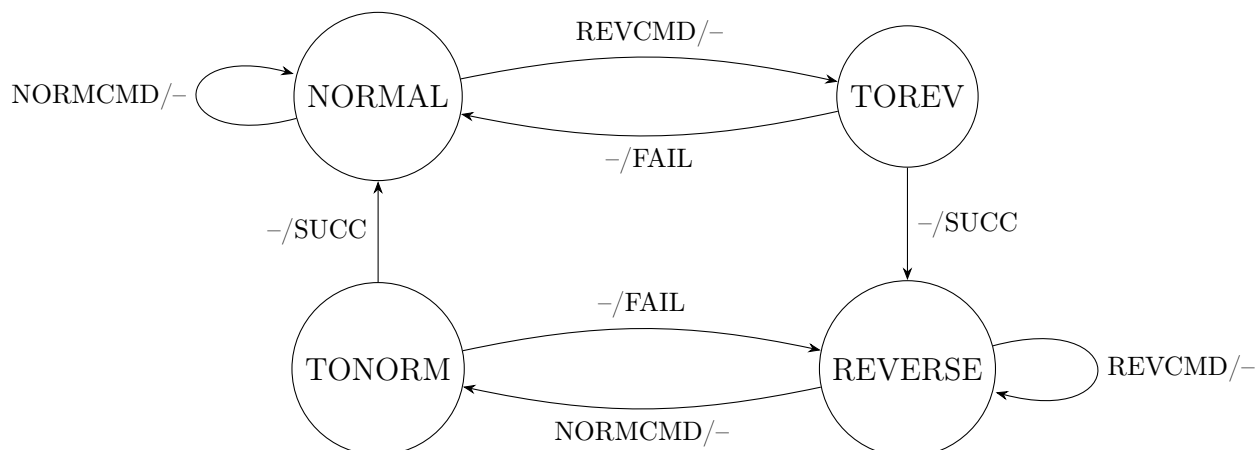


Figura 2: Macchina a stati SCAMBIO

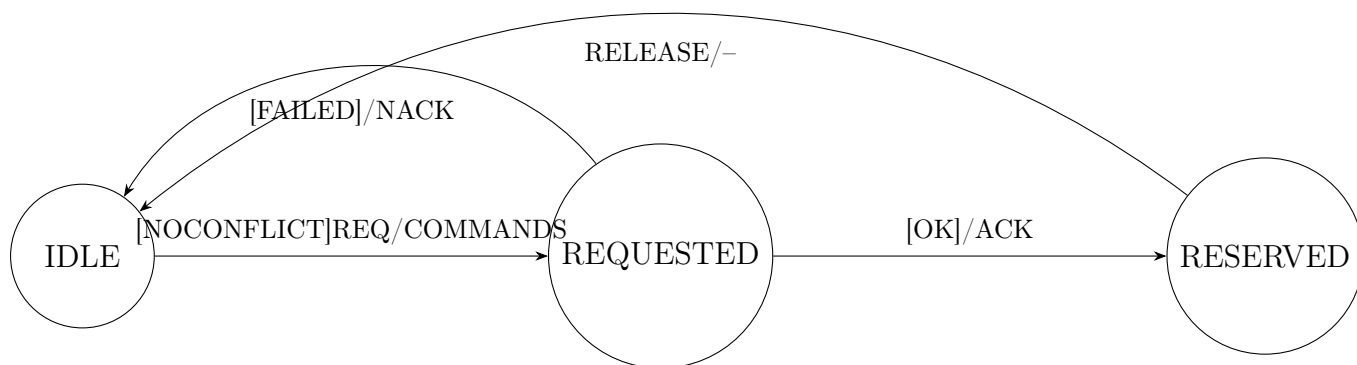


Figura 3: Macchina a stati ITINERARIO

1.5 Connessione con la mutua esclusione

Il sistema di interlocking è un'istanza del classico problema della **mutua esclusione**: gli itinerari conflittuali sono processi che non possono trovarsi simultaneamente in sezione critica (RESERVED).

Concetto di mutua esclusione	Nel sistema ferroviario
Sezione non critica	IDLE
Richiesta di accesso	IDLE → REQUESTED
Sezione critica	RESERVED
Rilascio	RESERVED → IDLE
Risorsa condivisa	Scambi fisici

2 Modellazione delle macchine a stati generiche

Effort stimato: 14 ore

2.1 Macchina scambio

La macchina a stati dello scambio modella il motore di posizionamento di ciascuno dei quattro scambi. Sono stati identificati quattro stati:

- NORMAL: scambio in posizione normale (diritta);
- TOREV: transizione verso la posizione rovescia (motore in moto);
- REVERSE: scambio in posizione rovescia;
- TONORM: transizione verso la posizione normale (motore in moto).

L'esito del motore (SUCC o FAIL) è modellato come variabile di input non deterministica (IVAR), a rappresentare il comportamento non controllabile del motore fisico.

Listing 1: Modulo scambio

```

1 MODULE scambio(revcmd, normcmd)
2 VAR
3   state : {NORMAL, TOREV, REVERSE, TONORM};
4 IVAR
5   outcome : {SUCC, FAIL};
6 ASSIGN
7   init(state) := NORMAL;
8   next(state) := case
9     state = NORMAL & revcmd           : TOREV;
10    state = REVERSE & normcmd          : TONORM;
11    state = TOREV & outcome = SUCC     : REVERSE;
12    state = TOREV & outcome = FAIL    : NORMAL;
13    state = TONORM & outcome = SUCC    : NORMAL;
14    state = TONORM & outcome = FAIL    : REVERSE;
15    TRUE                               : state;
16   esac;
```

Scelta progettuale: l'uso di IVAR per outcome permette di modellare il non determinismo del motore fisico senza introdurre variabili di stato aggiuntive, mantenendo il modello compatto.

2.2 Macchina itinerario

La macchina a stati dell'itinerario modella il ciclo di vita di una prenotazione: IDLE → REQUESTED → RESERVED → IDLE.

I parametri di ingresso stable e failed sono calcolati nel modulo principale e passati come segnali, disaccoppiando la logica dell'itinerario dalla conoscenza diretta degli scambi.

Listing 2: Modulo itinerario

```

1 MODULE itinerario(req, release, noconflict, failed, stable)
2 VAR
```

```

3   state : {IDLE, REQUESTED, RESERVED};
4  ASSIGN
5   init(state) := IDLE;
6   next(state) := case
7     state = IDLE      & req & noconflict      : REQUESTED;
8     state = REQUESTED & stable & !failed      : RESERVED;
9     state = REQUESTED & stable & failed       : IDLE;
10    state = RESERVED  & release                : IDLE;
11    TRUE              : state;
12  esac;

```

Scelta progettuale: la transizione REQUESTED $\rightarrow$ IDLE (fallimento) avviene solo quando `stable = TRUE`, ovvero quando tutti gli scambi hanno completato il posizionamento (con esito positivo o negativo). Ciò evita che l'itinerario reagisca a stati intermedi degli scambi.

3 Istanziatura e collegamento I/O

Effort stimato: 24 ore

3.1 Struttura del modulo principale

Il modulo `main` istanzia le otto macchine a stati (quattro scambi e quattro itinerari) e ne collega gli ingressi/uscite tramite segnali `DEFINE`.

Listing 3: Istanziatura nel modulo `main` (estratto)

```

1  MODULE main
2  VAR
3    s1 : scambio(s1_revcmd, s1_normcmd);
4    s2 : scambio(s2_revcmd, s2_normcmd);
5    s3 : scambio(s3_revcmd, s3_normcmd);
6    s4 : scambio(s4_revcmd, s4_normcmd);
7    AB : itinerario(req_AB, rel_AB, noconflict_AB, failed_AB,
8                  stable_AB);
9    AD : itinerario(req_AD, rel_AD, noconflict_AD, failed_AD,
10                  stable_AD);
11   CB : itinerario(req_CB, rel_CB, noconflict_CB, failed_CB,
12                  stable_CB);
13   CD : itinerario(req_CD, rel_CD, noconflict_CD, failed_CD,
14                  stable_CD);
15  IVAR
16    req_AB, req_AD, req_CB, req_CD : boolean;
17    rel_AB, rel_AD, rel_CB, rel_CD : boolean;

```

3.2 Condizione NOCONFLICT

La condizione `NOCONFLICT` di un itinerario è vera quando tutti gli itinerari in conflitto con esso si trovano nello stato `IDLE`:

Listing 4: Definizione `NOCONFLICT`

```

1 DEFINE
2   noconflict_AB := AD.state = IDLE & CB.state = IDLE;
3   noconflict_AD := AB.state = IDLE & CB.state = IDLE & CD.state =
      IDLE;
4   noconflict_CB := AB.state = IDLE & AD.state = IDLE & CD.state =
      IDLE;
5   noconflict_CD := AD.state = IDLE & CB.state = IDLE;

```

3.3 Azione COMMANDS

L'azione COMMANDS è modellata come un impulso di un passo che si attiva esattamente quando un itinerario transisce da IDLE a REQUESTED. Da questo segnale derivano i comandi ai singoli scambi:

Listing 5: Derivazione dei segnali di comando agli scambi

```

1 DEFINE
2   cmd_AB := AB.state = IDLE & req_AB & noconflict_AB;
3   cmd_AD := AD.state = IDLE & req_AD & noconflict_AD;
4   cmd_CB := CB.state = IDLE & req_CB & noconflict_CB;
5   cmd_CD := CD.state = IDLE & req_CD & noconflict_CD;
6
7   -- s1: AB->NORMALE, AD->NORMALE, CB->ROVESCIO
8   s1_normcmd := cmd_AB | cmd_AD;
9   s1_revcmd  := cmd_CB;
10  -- s2: AB->NORMALE, AD->ROVESCIO, CB->NORMALE
11  s2_normcmd := cmd_AB | cmd_CB;
12  s2_revcmd  := cmd_AD;
13  -- s3: CB->ROVESCIO, CD->NORMALE
14  s3_normcmd := cmd_CD;
15  s3_revcmd  := cmd_CB;
16  -- s4: AD->ROVESCIO, CD->NORMALE
17  s4_normcmd := cmd_CD;
18  s4_revcmd  := cmd_AD;

```

3.4 Condizioni STABLE e FAILED

STABLE è vera quando tutti gli scambi usati dall'itinerario si trovano in uno stato stabile (NORMAL o REVERSE). FAILED è vera quando almeno uno scambio non ha raggiunto la posizione bersaglio:

Listing 6: Condizioni STABLE e FAILED (itinerario AB)

```

1   stable_AB := (s1.state = NORMAL | s1.state = REVERSE) &
2               (s2.state = NORMAL | s2.state = REVERSE);
3
4   failed_AB := !(s1.state = NORMAL & s2.state = NORMAL);

```

4 Verifica della correttezza statica

Effort stimato: 3 ore

La correttezza statica è stata verificata tramite il comando `check_fsm` di NuXmv, che controlla l'assenza di stati di deadlock (stati privi di successori).

4.1 Risultato

The transition relation is total: No deadlock state exists

4.2 Motivazione

Il risultato è atteso per le seguenti ragioni:

- La macchina `scambio` ha sempre almeno una transizione: negli stati stabili (`NORMAL`, `REVERSE`) resta in se stessa se non riceve comandi; negli stati transitori (`TOREV`, `TONORM`) transisce in base all'esito `SUCC/FAIL`, che copre tutti i casi.
- La macchina `itinerario` ha sempre la transizione `TRUE: state` che mantiene lo stato corrente in assenza di condizioni attive.

5 Simulazione del modello

Effort stimato: 13 ore

Sono stati progettati ed eseguiti quattro scenari di simulazione, ciascuno finalizzato a illustrare un aspetto diverso del comportamento del sistema.

5.1 Scenario 1 – Sequenza base: AD poi AB (riuso degli scambi)

Questo scenario mostra l'itinerario AD che prenota (posizionando s2 e s4 in `REVERSE`), rilascia, e poi l'itinerario AB che prenota riportando s2 in `NORMAL`.

Stato	AD	AB	s2	Evento
1.1	IDLE	IDLE	NORMAL	Stato iniziale
1.2	REQUESTED	IDLE	TOREV	req_AD: comando inviato a s2, s4
1.3	REQUESTED	IDLE	REVERSE	s2, s4: SUCC
1.4	RESERVED	IDLE	REVERSE	stable & !failed: AD prenotata
1.5	IDLE	IDLE	REVERSE	rel_AD: AD rilasciata
1.6	IDLE	REQUESTED	TONORM	req_AB: s2 deve tornare NORMALE
1.7	IDLE	REQUESTED	NORMAL	s2: SUCC
1.8	IDLE	RESERVED	NORMAL	stable & !failed: AB prenotata
1.9	IDLE	IDLE	NORMAL	rel_AB: AB rilasciata

Tabella 2: Scenario 1: sequenza AD→AB

5.2 Scenario 2 – Fallimento di uno scambio e retry (CB)

Questo scenario dimostra la gestione del fallimento: s1 non riesce a posizionarsi in `REVERSE`, CB torna in `IDLE` e ripete la richiesta con successo al secondo tentativo.

Stato	CB	s1	Evento
1.1	IDLE	NORMAL	Stato iniziale
1.2	REQUESTED	TOREV	req_CB: comandi a s1, s3
1.3	REQUESTED	NORMAL	s1: FAIL (torna indietro), s3: SUCC
1.4	IDLE	NORMAL	failed_CB=TRUE: CB torna IDLE
1.5	REQUESTED	TOREV	Retry: solo s1 si riposiziona (s3 già OK)
1.6	REQUESTED	REVERSE	s1: SUCC
1.7	RESERVED	REVERSE	stable & !failed: CB prenotata
1.8	IDLE	REVERSE	Rilascio

Tabella 3: Scenario 2: fallimento s1 e retry di CB

5.3 Scenario 3 – Prenotazione parallela (AB || CD)

AB e CD non sono in conflitto e usano scambi disgiunti (s1,s2 vs s3,s4). Entrambi transitano a **RESERVED** nello stesso passo.

Stato	AB	CD	Evento
1.1	IDLE	IDLE	Stato iniziale
1.2	REQUESTED	REQUESTED	req_AB e req_CD simultanei
1.3	RESERVED	RESERVED	Entrambi prenotati nello stesso passo
1.4	IDLE	IDLE	Rilascio simultaneo

Tabella 4: Scenario 3: prenotazione parallela AB e CD

Questo scenario conferma che la condizione **NOCONFLICT** permette correttamente la coesistenza di itinerari non conflittuali.

5.4 Scenario 4 – AB bloccata da AD, poi servita

AB tenta di prenotare mentre AD è **RESERVED**: la condizione **noconflict_AB = FALSE** impedisce la transizione. Non appena AD rilascia, AB viene servita.

Stato	AB	AD	noconflict_AB	Evento
1.1	IDLE	IDLE	TRUE	Stato iniziale
1.2	IDLE	REQUESTED	FALSE	AD richiede, AB bloccata
1.3	IDLE	REQUESTED	FALSE	Scambi AD in transizione
1.4	IDLE	RESERVED	FALSE	AD prenotata, AB ancora bloccata
1.5	IDLE	RESERVED	FALSE	req_AB=TRUE ma noconflict=FALSE
1.6	IDLE	IDLE	TRUE	AD rilasciata, sblocco
1.7	REQUESTED	IDLE	TRUE	AB finalmente servita
1.8	REQUESTED	IDLE	TRUE	s2: TONORM → NORMAL
1.9	RESERVED	IDLE	TRUE	AB prenotata
1.10	IDLE	IDLE	TRUE	Rilascio

Tabella 5: Scenario 4: AB bloccata da AD, poi servita

6 Verifica formale delle proprietà

Effort stimato: 18 ore

Le proprietà sono state espresse in logica temporale CTL (per Safety-1, Safety-2, Liveness) e LTL (per Fairness), e verificate con il model checker di NuXmv.

6.1 Safety-1 (No-collision)

Proprietà: non è mai possibile che due itinerari in conflitto siano simultaneamente nello stato RESERVED.

Listing 7: Specifica CTL Safety-1

```

1 SPEC AG !(AB.state = RESERVED & AD.state = RESERVED)
2 SPEC AG !(AB.state = RESERVED & CB.state = RESERVED)
3 SPEC AG !(AD.state = RESERVED & CB.state = RESERVED)
4 SPEC AG !(AD.state = RESERVED & CD.state = RESERVED)
5 SPEC AG !(CB.state = RESERVED & CD.state = RESERVED)

```

Output NuXmv:

```

-- specification AG !(AB.state=RESERVED & AD.state=RESERVED) is true
-- specification AG !(AB.state=RESERVED & CB.state=RESERVED) is true
-- specification AG !(AD.state=RESERVED & CB.state=RESERVED) is true
-- specification AG !(AD.state=RESERVED & CD.state=RESERVED) is true
-- specification AG !(CB.state=RESERVED & CD.state=RESERVED) is true

```

Risultato: vera per tutte e cinque le coppie conflittuali.

Argomento informale: per transitare in RESERVED, un itinerario X deve avere `noconflict_X = TRUE`, ovvero tutti i conflittuali in IDLE. Una volta che X è in REQUESTED o RESERVED, i conflittuali hanno `noconflict = FALSE` e non possono avanzare. Inoltre, i conflittuali richiedono posizioni degli scambi fisicamente incompatibili (es. AB vuole `s1=NORMAL`, CB vuole `s1=REVERSE`), rendendo impossibile che entrambi raggiungano RESERVED simultaneamente.

6.2 Safety-2 (No-derailment)

Proprietà: gli scambi di un itinerario in stato RESERVED non ricevono comandi di posizionamento (non si trovano in stati di transizione).

Listing 8: Specifica CTL Safety-2 (formulazione equivalente senza IVAR)

```

1 SPEC AG (AB.state = RESERVED ->
2     s1.state != TOREV & s1.state != TONORM &
3     s2.state != TOREV & s2.state != TONORM)

```

Nota sulla formulazione: poiché i segnali di comando (`s1_normcmd`, ecc.) dipendono da variabili IVAR non ammesse nelle specifiche CTL, la proprietà è stata riformulata verificando che gli scambi non si trovino in stati di transizione (TOREV/TONORM) mentre l'itinerario è RESERVED. Le due formulazioni sono equivalenti: uno scambio entra in stato di transizione solo se riceve un comando.

Output NuXmv:

```
-- specification AG (AB.state=RESERVED ->
    s1.state!=TOREV & s1.state!=TONORM &
    s2.state!=TOREV & s2.state!=TONORM) is true
-- specification AG (AD.state=RESERVED -> ... s4.state!=TONORM) is true
-- specification AG (CB.state=RESERVED -> ... s3.state!=TONORM) is true
-- specification AG (CD.state=RESERVED ->
    s3.state!=TOREV & s3.state!=TONORM &
    s4.state!=TOREV & s4.state!=TONORM) is true
```

Risultato: vera per tutti e quattro gli itinerari.

Argomento informale: i comandi agli scambi sono emessi solo quando un itinerario transisce da IDLE a REQUESTED (`cmd_X = X.state = IDLE ...`). Se X è RESERVED, allora per ogni itinerario Y che usa gli stessi scambi, Y è un conflittuale di X, e `noconflict_Y = FALSE` (perché $X \neq \text{IDLE}$), quindi `cmd_Y = FALSE`. Nessun altro itinerario emette comandi agli scambi di X.

6.3 Liveness (possibilità di prenotazione)

Proprietà: per ogni itinerario in stato REQUESTED, esiste sempre un cammino che porta a RESERVED.

Listing 9: Specifica CTL Liveness

```
1 SPEC AG (AB.state = REQUESTED -> EF AB.state = RESERVED)
2 SPEC AG (AD.state = REQUESTED -> EF AD.state = RESERVED)
3 SPEC AG (CB.state = REQUESTED -> EF CB.state = RESERVED)
4 SPEC AG (CD.state = REQUESTED -> EF CD.state = RESERVED)
```

Output NuXmv:

```
-- specification AG (AB.state=REQUESTED -> EF AB.state=RESERVED) is true
-- specification AG (AD.state=REQUESTED -> EF AD.state=RESERVED) is true
-- specification AG (CB.state=REQUESTED -> EF CB.state=RESERVED) is true
-- specification AG (CD.state=REQUESTED -> EF CD.state=RESERVED) is true
```

Risultato: vera per tutti e quattro gli itinerari.

Argomento informale: dallo stato REQUESTED, esiste sempre un cammino in cui tutti gli scambi ricevono esito SUCC, raggiungono la posizione bersaglio, e l'itinerario transisce in RESERVED. Questa è una proprietà esistenziale (EF): non è necessario che avvenga sempre, ma che sia sempre possibile.

6.4 Fairness (assenza di ritardo indefinito)

Proprietà: sotto opportune assunzioni di fairness, ogni itinerario in IDLE arriva prima o poi in RESERVED.

Le assunzioni di fairness aggiunte al modello sono:

- FAIRNESS `si.outcome = SUCC` (ogni scambio riesce prima o poi);
- FAIRNESS `rel_X` (ogni itinerario rilascia prima o poi);
- FAIRNESS `req_X` (ogni itinerario richiede infinite volte).

Listing 10: Specifica LTL Fairness

```

1 LTLSPEC G (AB.state = IDLE -> F AB.state = RESERVED)
2 LTLSPEC G (AD.state = IDLE -> F AD.state = RESERVED)
3 LTLSPEC G (CB.state = IDLE -> F CB.state = RESERVED)
4 LTLSPEC G (CD.state = IDLE -> F CD.state = RESERVED)

```

Output NuXmv:

```

-- specification G (AB.state=IDLE -> F AB.state=RESERVED) is false
-- as demonstrated by the following execution sequence
-- specification G (AD.state=IDLE -> F AD.state=RESERVED) is false
-- specification G (CB.state=IDLE -> F CB.state=RESERVED) is false
-- specification G (CD.state=IDLE -> F CD.state=RESERVED) is false

```

Risultato: falsa per tutti e quattro gli itinerari.

6.4.1 Analisi del controesempio

Il model checker ha prodotto un controesempio (traccia per AB) che mostra un'esecuzione *fair* in cui AB rimane in IDLE indefinitamente. Di seguito i passi chiave della traccia:

```

-- Loop starts here
-> State: 1.1 <-          -- tutti IDLE, scambi NORMAL
    AB.state = IDLE,  AD.state = IDLE

-> Input: 1.5 <-          -- AB e AD richiedono contemporaneamente
    req_AB = TRUE,  req_AD = TRUE

-> State: 1.6 <-          -- entrambi REQUESTED; s2 va in TOREV (AD vince)
    AB.state = REQUESTED,  AD.state = REQUESTED
    s2.state = TOREV,  s4.state = TOREV

-> State: 1.7 <-          -- scambi si stabilizzano in REVERSE (per AD)
    s2.state = REVERSE,  s4.state = REVERSE

-> State: 1.8 <-          -- AD->RESERVED; AB->IDLE (failed_AB=TRUE)
    AB.state = IDLE,  AD.state = RESERVED

```

Il meccanismo di starvation è il seguente:

1. AB e AD richiedono simultaneamente (entrambi soddisfano NOCONFLICT perché sono ancora IDLE nel momento della valutazione).
2. I comandi a s2 sono conflittuali: AD richiede REVERSE, AB richiede NORMAL. Il caso revcmd ha priorità nella case expression $\Rightarrow$ s2 va in TOREV.
3. Gli scambi si stabilizzano nella posizione corretta per AD: AD $\rightarrow$ RESERVED, AB $\rightarrow$ IDLE (failed_AB = TRUE).
4. AD (o un altro conflittuale) richiede immediatamente di nuovo, vincendo ogni volta la competizione.
5. Il ciclo si ripete indefinitamente: AB è sempre IDLE ma non viene mai servita.

6.4.2 Causa del fallimento: sistema senza memoria

La condizione NOCONFLICT è *puramente reattiva e senza memoria*: si limita a controllare lo stato corrente dei conflittuali, senza tenere conto di quanto a lungo un itinerario stia aspettando. Il meccanismo è analogo al **test-and-set** nella teoria della mutua esclusione, che notoriamente non garantisce starvation-freedom in assenza di ulteriori meccanismi.

6.4.3 Osservazione

La falsità della proprietà di Fairness è un risultato corretto che riflette una **limitazione intrinseca della specifica**: il sistema non è progettato per garantire equità di servizio. Nei sistemi di interlocking reali, questo problema è affrontato a un livello superiore (controllore del traffico ferroviario) mediante meccanismi di priorità o code FIFO. Una possibile soluzione nel modello sarebbe l'aggiunta di variabili di attesa (es. timestamp di richiesta) per implementare un ordinamento con priorità – tema che esula però dalla specifica del progetto.

7 Riepilogo dell'effort

#	Obiettivo	Ore
1	Modellazione macchine a stati generiche (<i>scambio</i> , <i>itinerario</i>)	14
2	Istanziamento, collegamento I/O, NOCONFLICT, COMMANDS, FAILED	24
3	Verifica correttezza statica (<i>check_fsm</i>)	3
4	Simulazione (4 scenari)	13
5	Verifica formale con model checking e discussione risultati	18
Totale		72

Tabella 6: Riepilogo dell'effort per obiettivo

8 Conclusioni

Il sistema di interlocking ferroviario è stato modellato con successo in NuXmv mediante due moduli generici (*scambio* e *itinerario*) istanziati rispettivamente quattro volte ciascuno e collegati tramite segnali derivati nel modulo principale.

La verifica formale ha prodotto i seguenti risultati:

- **Safety-1** (no-collision): **verificata**. Nessuna coppia di itinerari conflittuali può trovarsi simultaneamente in *RESERVED*.
- **Safety-2** (no-derailment): **verificata**. Gli scambi di un itinerario prenotato non ricevono comandi di posizionamento.
- **Liveness**: **verificata**. Ogni itinerario in *REQUESTED* ha sempre la possibilità di raggiungere *RESERVED*.
- **Fairness**: **non verificata**. Il sistema, così come specificato, non garantisce l'assenza di starvation. Il controesempio prodotto dal model checker è un'esecuzione fair

in cui un itinerario viene indefinitamente battuto da un conflittuale, analogamente a quanto accade con il test-and-set nella teoria della mutua esclusione.

Il progetto ha permesso di applicare concretamente le tecniche di modellazione formale e model checking a un sistema safety-critical, evidenziando sia le proprietà garantite dalla specifica sia i suoi limiti, in particolare l'assenza di un meccanismo di equità.